

Complement Formulas

Derived from `src/utils/complement.ts`.

How Hex Color Codes Work

A hex color code is just a compact way to pack three numbers into one string — one number each for red, green, and blue light intensity.

Base-16 (hexadecimal)

You're probably used to base 10, which uses digits 0–9. Hexadecimal (base 16) extends that with six more: A=10, B=11, C=12, D=13, E=14, F=15.

With two hex digits, you can express any value from `00` to `FF`:

$$00_{16} = 0 \times 16 + 0 = 0_{10}$$

$$FF_{16} = 15 \times 16 + 15 = 255_{10}$$

Two hex digits give you exactly 256 possible values — 2^8 , or one byte. That's the full range `[0, 255]` for a single color channel.

Structure of a hex code

A 6-character hex code is just three 2-digit hex numbers concatenated together, one per channel:

`#RRGGBB`

`#FF5733`

—	—	—	
FF	57	33	
			Blue = 0x33 = 51
	└─		Green = 0x57 = 87
└─			Red = 0xFF = 255

Each channel goes from `00` (off) to `FF` (full blast). Mix all three and you get your color — this is additive color mixing, the same way your monitor works.

RGB as a 3D cube

It helps to think of RGB as a point in a cube. Each axis is one channel, spanning $[0, 255]$:

- $(0, 0, 0) \rightarrow$ black
- $(255, 255, 255) \rightarrow$ white
- $(255, 0, 0) \rightarrow$ pure red

Every color is a point somewhere in that cube. The complement operations below are just geometric moves on that point — shifts, reflections, inversions.

HSL: a more human-friendly view

RGB is great for machines but a bit awkward for humans. HSL remaps the same colors onto a cylinder, which maps better to how we actually think about color:

Component	Range	What it controls
Hue H	$[0^\circ, 360^\circ)$	The "color" — your position on the color wheel ($0^\circ =$ red, $120^\circ =$ green, $240^\circ =$ blue)
Saturation S	$[0, 1]$	How vivid — 0 is gray, 1 is full color
Lightness L	$[0, 1]$	How bright — 0 is black, 0.5 is the pure hue, 1 is white

Think of it this way: hue is the **angle** around the cylinder, saturation is the **distance from the center**, and lightness is the **height**. Taking a complement in this space is just reflecting across one or more of those axes.

HSL Complements

All the mirror variants below are built from three base complement values:

Component	Original	Complement
Hue	H	$H' = 360 - H$
Saturation	S	$S' = 1 - S$
Lightness	L	$L' = 1 - L$

Quick note on $H' = 360 - H$: this reflects the hue across the $0^\circ/360^\circ$ boundary on the color wheel. It's not the same as $H + 180^\circ$, which would give you the color directly opposite — these are two different ideas of "opposite hue."

Mirror Set Variants

`getMirrorSet` generates all 7 combinations of swapping original vs. complement values across H, S, and L. The naming convention: **uppercase = use the complement**, lowercase = keep the original.

Key	Hue	Saturation	Lightness	Formula
<code>mirror_HSL</code>	H'	S'	L'	$(360 - H, 1 - S, 1 - L)$
<code>mirror_HSl</code>	H'	S'	L	$(360 - H, 1 - S, L)$
<code>mirror_HsL</code>	H'	S	L'	$(360 - H, S, 1 - L)$
<code>mirror_Hsl</code>	H'	S	L	$(360 - H, S, L)$
<code>mirror_hSL</code>	H	S'	L'	$(H, 1 - S, 1 - L)$
<code>mirror_hSl</code>	H	S'	L	$(H, 1 - S, L)$
<code>mirror_hsL</code>	H	S	L'	$(H, S, 1 - L)$

Google Sheets Complement

`getGSheetsComp` recreates the dark/light pairing behavior you get in Google Sheets — where a color is automatically paired with a lighter or darker version of itself.

At module load, a fixed delta K is picked once and reused:

$$K = 113 + \text{round}(\text{random}() \times 4), \quad K \in \{113, 114, 115, 116, 117\}$$

Whether that delta gets added or subtracted depends on the color's lightness:

$$\delta = \begin{cases} +K & \text{if } L < 0.5 \quad (\text{dark color, shift lighter}) \\ -K & \text{if } L \geq 0.5 \quad (\text{light color, shift darker}) \end{cases}$$

That same delta is applied to all three RGB channels, then clamped to stay in range:

$$R' = \text{clamp}(R + \delta, 0, 255)$$

$$G' = \text{clamp}(G + \delta, 0, 255)$$

$$B' = \text{clamp}(B + \delta, 0, 255)$$

where $\text{clamp}(x, a, b) = \max(a, \min(b, x))$.

Because the shift is the same across all three channels, hue and saturation stay roughly intact — you just get a brighter or darker version of the original color.

Midpoint Complement

`getMidpointComp` is the simplest one: flip each channel around the midpoint of its range.

The midpoint of $[0, 255]$ is 127.5, so reflecting a value x across it gives $255 - x$:

$$R' = 255 - R$$

$$G' = 255 - G$$

$$B' = 255 - B$$

Geometrically, this is a point reflection through the center of the RGB cube — (127.5, 127.5, 127.5). In plain terms, it's the photographic negative of the input color.

Summary

Variant	Color Space	Operation
<code>mirror_Hsl</code>	HSL	Hue reflected across 0°
<code>mirror_hSl</code>	HSL	Saturation inverted
<code>mirror_hsL</code>	HSL	Lightness inverted
<code>mirror_HSL</code>	HSL	Hue + Saturation reflected/inverted
<code>mirror_hSL</code>	HSL	Hue + Lightness reflected/inverted
<code>mirror_hSL</code>	HSL	Saturation + Lightness inverted
<code>mirror_HSL</code>	HSL	All three channels reflected/inverted
<code>mirror_gsheet</code>	RGB	Uniform $\pm K$ shift based on luminance
<code>mirror_midpoint</code>	RGB	Point reflection through (127.5, 127.5, 127.5)